

Los siguientes comandos detallan la configuración de PostgreSQL considerando un entorno de sistema operativo CentOS 9.

Shell

```
# Conéctate a tu instancia en GCP
gcloud compute ssh tu-instancia-postgres --zone=tu-zona
# Instala postgres contrib para pgcrypto
sudo dnf install postgresql-contrib
# Cambia al usuario postgres y entra a la consola
sudo -i -u postgres
psql
-- Crea base de datos y conecta a ella
CREATE DATABASE retail_perecederos;
\c retail_perecederos
-- Instala extensiones
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
-- Crea esquemas
CREATE SCHEMA operacion;
CREATE SCHEMA auditoria;
-- Revoca acceso publico predeterminado por seguridad
REVOKE ALL ON DATABASE retail_perecederos FROM PUBLIC;
REVOKE ALL ON SCHEMA public FROM PUBLIC;
-- Rol transaccional para api o conexion
CREATE ROLE app_backend WITH LOGIN PASSWORD '999';
CREATE ROLE app_auditor WITH LOGIN PASSWORD '999';
-- Permisos de conexion
GRANT CONNECT ON DATABASE retail_perecederos TO app_backend, app_auditor;
GRANT USAGE ON SCHEMA operacion TO app_backend;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA operacion TO app_backend;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA operacion TO app_backend;
GRANT USAGE ON SCHEMA auditoria TO app_backend;
GRANT INSERT ON ALL TABLES IN SCHEMA auditoria TO app_backend;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA auditoria TO app_backend;
GRANT USAGE ON SCHEMA operacion, auditoria TO app_auditor;
GRANT SELECT ON ALL TABLES IN SCHEMA operacion TO app_auditor;
GRANT SELECT ON ALL TABLES IN SCHEMA auditoria TO app_auditor;
ALTER DEFAULT PRIVILEGES IN SCHEMA operacion GRANT ALL ON TABLES TO
app_backend;
ALTER DEFAULT PRIVILEGES IN SCHEMA auditoria GRANT INSERT ON TABLES TO
app_backend;
ALTER DEFAULT PRIVILEGES IN SCHEMA operacion, auditoria GRANT SELECT ON TABLES
TO app_auditor;
\q
# sal de usuario postgres
exit
# edita archivo de configuracion
vim /var/lib/pgsql/data/pg_hba.conf
```

```
# Agrega al final (reemplaza por la IP de tu backend o 0.0.0.0/0 para
desarrollo): # host retail_perecederos app_backend 0.0.0.0/0 scram-sha-256 #
host retail_perecederos app_auditor 0.0.0.0/0 scram-sha-256
sudo systemctl restart postgresql
```

SCRIPT DDL PARA GENERAR TABLAS DE OPERACIÓN

SQL

```
-- Establecer el esquema por defecto para esta sesión
SET search_path TO operacion;

--
=====
=====
-- 1. CATÁLOGOS MAESTROS Y SEGURIDAD (Sin dependencias externas)
--
=====
=====

CREATE TABLE rol (
    id_rol INT GENERATED ALWAYS AS IDENTITY,
    nombre_rol VARCHAR(50) NOT NULL,
    CONSTRAINT pk_rol PRIMARY KEY (id_rol),
    CONSTRAINT uq_nombre_rol UNIQUE (nombre_rol)
);

CREATE TABLE locacion (
    id_locacion INT GENERATED ALWAYS AS IDENTITY,
    tipo_locacion VARCHAR(20) NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    CONSTRAINT pk_locacion PRIMARY KEY (id_locacion),
    CONSTRAINT chk_tipo_locacion CHECK (tipo_locacion IN
('Tienda', 'CEDIS'))
);

CREATE TABLE proveedor (
    id_proveedor INT GENERATED ALWAYS AS IDENTITY,
    rfc CHAR(13) NOT NULL,
```

```

    razon_social VARCHAR(150) NOT NULL,
    lead_time_dias INT NOT NULL,
    CONSTRAINT pk_proveedor PRIMARY KEY (id_proveedor),
    CONSTRAINT uq_rfc UNIQUE (rfc),
    CONSTRAINT chk_lead_time CHECK (lead_time_dias >= 0)
);

```

```

CREATE TABLE producto (
    id_sku INT GENERATED ALWAYS AS IDENTITY,
    codigo_gtin CHAR(14) NOT NULL,
    nombre VARCHAR(150) NOT NULL,
    vida_util_estandar INT NOT NULL,
    CONSTRAINT pk_producto PRIMARY KEY (id_sku),
    CONSTRAINT uq_codigo_gtin UNIQUE (codigo_gtin),
    CONSTRAINT chk_vida_util CHECK (vida_util_estandar > 0)
);

```

```
--
```

```

=====
=====

```

```
-- 2. TABLAS CON DEPENDENCIAS DE PRIMER NIVEL
```

```
--
```

```

=====
=====

```

```

CREATE TABLE usuario (
    id_usuario UUID DEFAULT gen_random_uuid(),
    id_rol INT NOT NULL,
    id_locacion INT NOT NULL,
    nombre_completo VARCHAR(150) NOT NULL,
    estado_activo BOOLEAN DEFAULT true NOT NULL,
    CONSTRAINT pk_usuario PRIMARY KEY (id_usuario),
    CONSTRAINT fk_usuario_rol FOREIGN KEY (id_rol)
        REFERENCES rol(id_rol) ON DELETE RESTRICT,
    CONSTRAINT fk_usuario_locacion FOREIGN KEY (id_locacion)
        REFERENCES locacion(id_locacion) ON DELETE RESTRICT
);

```

```
CREATE TABLE lote (
```

```

    id_lote UUID DEFAULT gen_random_uuid(),
    id_sku INT NOT NULL,
    id_proveedor INT NOT NULL,
    codigo_lote_prov VARCHAR(50) NOT NULL,
    fecha_caducidad DATE NOT NULL,
    CONSTRAINT pk_lote PRIMARY KEY (id_lote),
    CONSTRAINT fk_lote_producto FOREIGN KEY (id_sku)
        REFERENCES producto(id_sku) ON DELETE RESTRICT,
    CONSTRAINT fk_lote_proveedor FOREIGN KEY (id_proveedor)
        REFERENCES proveedor(id_proveedor) ON DELETE RESTRICT
);

--
=====
=====
-- 3. NÚCLEO OPERATIVO Y TRANSACCIONAL (Dependencias de segundo
nivel)
--
=====
=====

CREATE TABLE existencia (
    id_existencia UUID DEFAULT gen_random_uuid(),
    id_lote UUID NOT NULL,
    id_locacion INT NOT NULL,
    cantidad_disponible DECIMAL(10,2) NOT NULL,
    cantidad_reservada DECIMAL(10,2) DEFAULT 0 NOT NULL,
    CONSTRAINT pk_existencia PRIMARY KEY (id_existencia),
    CONSTRAINT fk_existencia_lote FOREIGN KEY (id_lote)
        REFERENCES lote(id_lote) ON DELETE RESTRICT,
    CONSTRAINT fk_existencia_locacion FOREIGN KEY (id_locacion)
        REFERENCES locacion(id_locacion) ON DELETE RESTRICT,
    CONSTRAINT chk_cantidad_disponible CHECK (cantidad_disponible
>= 0),
    CONSTRAINT chk_cantidad_reservada CHECK (cantidad_reservada
>= 0)
);

CREATE TABLE venta_detalle (

```

```

    id_venta UUID DEFAULT gen_random_uuid(),
    id_lote UUID NOT NULL,
    id_locacion INT NOT NULL,
    cantidad DECIMAL(10,2) NOT NULL,
    fecha_transaccion TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT
NULL,
    CONSTRAINT pk_venta_detalle PRIMARY KEY (id_venta),
    CONSTRAINT fk_venta_lote FOREIGN KEY (id_lote)
        REFERENCES lote(id_lote) ON DELETE RESTRICT,
    CONSTRAINT fk_venta_locacion FOREIGN KEY (id_locacion)
        REFERENCES locacion(id_locacion) ON DELETE RESTRICT,
    CONSTRAINT chk_cantidad_venta CHECK (cantidad > 0)
);

```

```

CREATE TABLE merma (
    id_merma UUID DEFAULT gen_random_uuid(),
    id_lote UUID NOT NULL,
    id_usuario UUID NOT NULL,
    cantidad DECIMAL(10,2) NOT NULL,
    causa_merma VARCHAR(30) NOT NULL,
    fecha_registro TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT
NULL,
    CONSTRAINT pk_merma PRIMARY KEY (id_merma),
    CONSTRAINT fk_merma_lote FOREIGN KEY (id_lote)
        REFERENCES lote(id_lote) ON DELETE RESTRICT,
    CONSTRAINT fk_merma_usuario FOREIGN KEY (id_usuario)
        REFERENCES usuario(id_usuario) ON DELETE RESTRICT,
    CONSTRAINT chk_cantidad_merma CHECK (cantidad > 0)
);

```

SCRIPT DDL PARA GENERAR TABLAS DE AUDITORÍA

1. Crear la tabla de Bitácora Centralizada:

Esta tabla residirá en el esquema auditoria y almacenará la evidencia inalterable. Utiliza JSONB para absorber cualquier cambio sin necesidad de modificar su estructura si en el futuro agregas columnas a las tablas operativas.

SQL

```
SET search_path TO auditoria;
```

```
CREATE TABLE bitacora_eventos (  
    id_evento UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
    nombre_tabla VARCHAR(100) NOT NULL,  
    tipo_operacion VARCHAR(10) NOT NULL CHECK (tipo_operacion IN  
( 'INSERT', 'UPDATE', 'DELETE' )),  
    id_usuario_app UUID, -- Identificador del usuario del sistema  
    (no el rol de BD)  
    estado_anterior JSONB,  
    estado_nuevo JSONB,  
    fecha_evento TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL  
);
```

```
CREATE OR REPLACE FUNCTION fn_registrar_auditoria()  
RETURNS TRIGGER AS $$  
DECLARE  
    v_user_id UUID;  
BEGIN  
    -- Captura el UUID del usuario real desde una variable de  
    sesión configurada por el backend  
    BEGIN  
        v_user_id := current_setting('app.current_user_id',  
true)::UUID;  
    EXCEPTION WHEN OTHERS THEN
```

```

        v_user_id := NULL; -- Si el administrador opera directo
en consola, quedará nulo
    END;

    IF (TG_OP = 'DELETE') THEN
        INSERT INTO auditoria.bitacora_eventos
            (nombre_tabla, tipo_operacion, id_usuario_app,
estado_anterior, estado_nuevo)
        VALUES
            (TG_TABLE_NAME, TG_OP, v_user_id,
row_to_json(OLD)::JSONB, NULL);
        RETURN OLD;

    ELSIF (TG_OP = 'UPDATE') THEN
        -- Optimización: Solo registrar si realmente hubo cambios
        IF (OLD IS DISTINCT FROM NEW) THEN
            INSERT INTO auditoria.bitacora_eventos
                (nombre_tabla, tipo_operacion, id_usuario_app,
estado_anterior, estado_nuevo)
            VALUES
                (TG_TABLE_NAME, TG_OP, v_user_id,
row_to_json(OLD)::JSONB, row_to_json(NEW)::JSONB);
            END IF;
            RETURN NEW;

        ELSIF (TG_OP = 'INSERT') THEN
            INSERT INTO auditoria.bitacora_eventos
                (nombre_tabla, tipo_operacion, id_usuario_app,
estado_anterior, estado_nuevo)
            VALUES
                (TG_TABLE_NAME, TG_OP, v_user_id, NULL,
row_to_json(NEW)::JSONB);
            RETURN NEW;
        END IF;

        RETURN NULL;
    END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

2.Desarrollar la función PL/pgSQL del Trigger:Motor de captura dinámica.

Esta función detecta qué operación se ejecutó (**TG_OP**) y empaqueta el estado de la fila en JSON. Además, captura el ID del usuario real desde el contexto de la sesión, resolviendo el problema de que todas las conexiones web usan el mismo rol técnico (**app_backend**).

```
SQL
CREATE OR REPLACE FUNCTION fn_bloquear_modificacion_auditoria()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'Violación de seguridad: No está permitido
    alterar o eliminar registros de la bitácora de auditoría.';
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_proteger_bitacora
BEFORE UPDATE OR DELETE ON auditoria.bitacora_eventos
FOR EACH ROW EXECUTE FUNCTION
fn_bloquear_modificacion_auditoria();
```

4.Acoplar el disparador a las entidades críticas: Vinculación a las tablas operativas.

Asocia la función de auditoría a las tablas del esquema **operacion** donde el impacto logístico o financiero es crítico.

```
SQL
SET search_path TO operacion;

-- Auditoría para movimientos de inventario (Crucial para FEF0)
CREATE TRIGGER trg_auditoria_existencia
AFTER INSERT OR UPDATE OR DELETE ON existencia
FOR EACH ROW EXECUTE FUNCTION auditoria.fn_registrar_auditoria();

-- Auditoría para bajas contables
CREATE TRIGGER trg_auditoria_merma
AFTER INSERT OR UPDATE OR DELETE ON merma
FOR EACH ROW EXECUTE FUNCTION auditoria.fn_registrar_auditoria();

-- Auditoría para salidas en punto de venta
CREATE TRIGGER trg_auditoria_venta
AFTER INSERT OR UPDATE OR DELETE ON venta_detalle
FOR EACH ROW EXECUTE FUNCTION auditoria.fn_registrar_auditoria();
```



```
-- Auditoría para gestión de lotes
CREATE TRIGGER trg_auditoria_lote
AFTER INSERT OR UPDATE OR DELETE ON lote
FOR EACH ROW EXECUTE FUNCTION auditoria.fn_registrar_auditoria();
```